Bicol University POLANGUI CAMPUS

Centro Occidental Polangui, Albay

IT112: WEB SYSTEMS and TECHNOLOGIE'S

2nd Semester, AY: 2025-2026

Phase 5: Final Group Project

BentaBoard

"From bawat benta to tunay na kita—track it right."

Proposed By:

PeNiHAMiNo

BSIT 2A

Mantes, Michael B.

Obidece, Hazel Ann R.

Odo, Pearl Anne R.

Racadag, Nica Alyssa R.

Remolar, Noe Jr H.

Presented to:

Dr. Guillermo V. Red, Jr.

IT112 Professor

Introduction

After successfully connecting our frontend to our backend in Phase 4, the next big challenge for **BentaBoard** was making the data actually visible and useful to the user. In a management system like ours, it isn't enough to just save data into a database; the user needs to be able to see their products, check their sales history, and monitor their expenses in real-time through a clean and organized interface. This is the core purpose of **Phase 5: Data Retrieval and Dynamic Display**.

In this phase, our group focused on "bringing the data to life." We moved away from static pages and worked on implementing logic that "asks" the server for information using the JavaScript **fetch()** API. Once the server (Node.js/Express) sends the data back in JSON format, our frontend uses DOM manipulation to automatically build tables, cards, and lists on the screen.

We specifically focused on creating a functional Dashboard and an Admin view. For the Seller, this means seeing a summary of their business health. For the Admin, it means having the ability to view and manage the users registered in the system. By automating this process, BentaBoard becomes a truly dynamic tool where a small business owner can log in and immediately see their current stock levels, recent sales, and total profits without any manual refreshing or hard-coded information. This phase turns our project from a collection of forms into a real, working business management dashboard.

Objectives

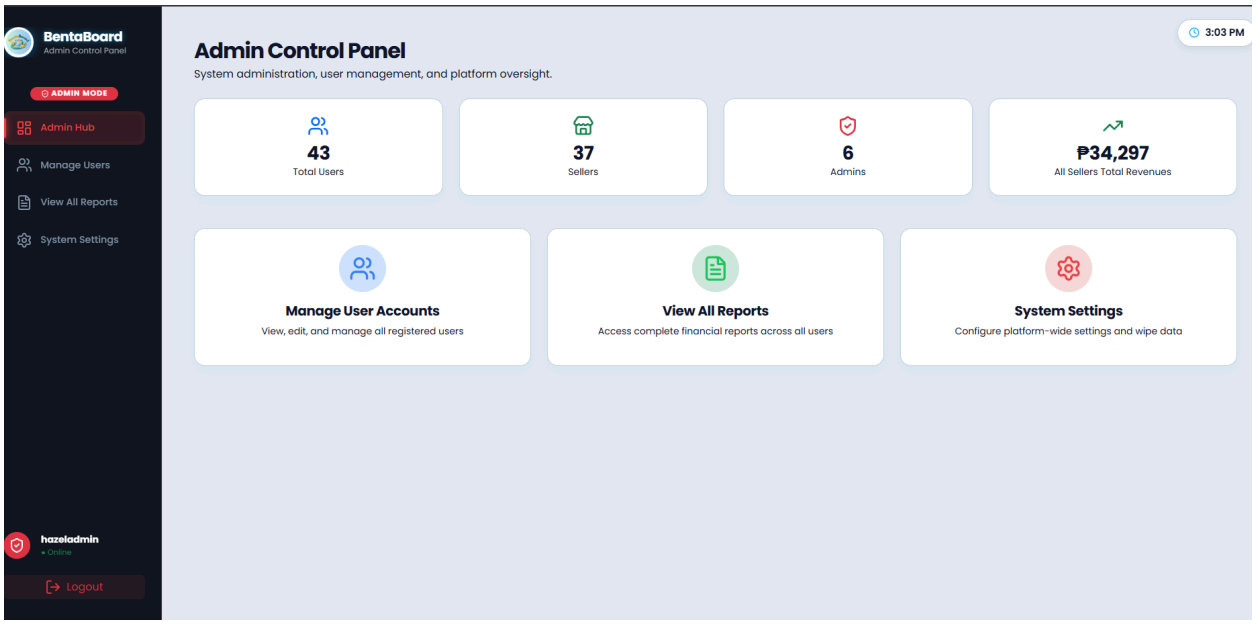
<u>At the end of this activity, we were able to:</u>

1. Retrieve data from backend APIs using fetch() or axios in the frontend.
2. Dynamically display content using DOM manipulation (e.g., tables, cards).
3. Design reusable templates for dashboard and user interfaces.
4. Implement simple pagination or content control for usability.
5. Manage GitHub commits and update frontend/backend structure accordingly.

I. Screenshot of at least 2 display pages

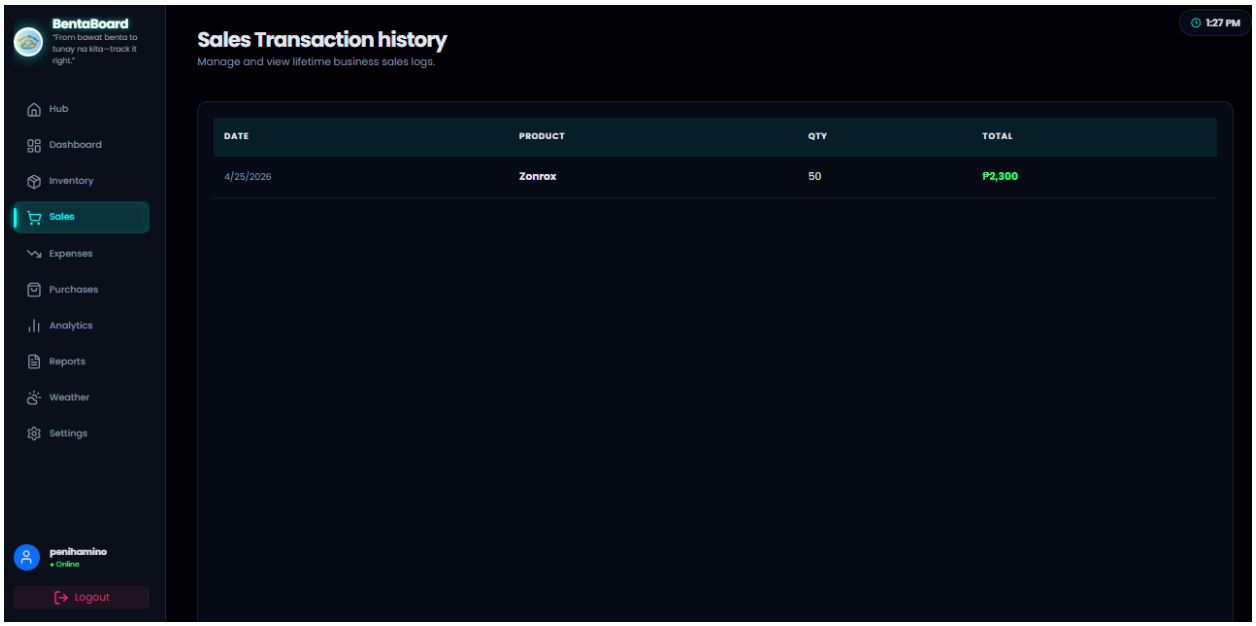
A. Admin User Management Page

As outlined in our proposal, the Admin has the authority to manage the system. This screenshot shows the list of registered users retrieved from the /api/users endpoint.



B. User Sales Tab

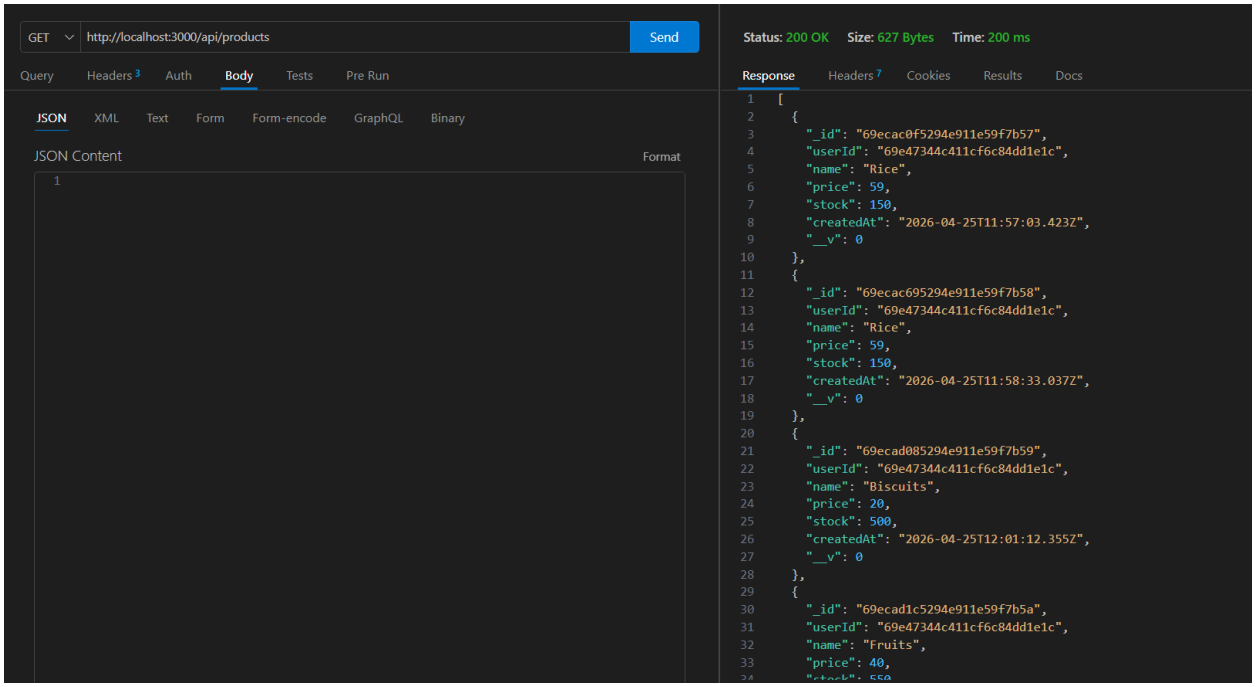
This page shows the dynamic rendering of products and recent sales data fetched from the MongoDB database.

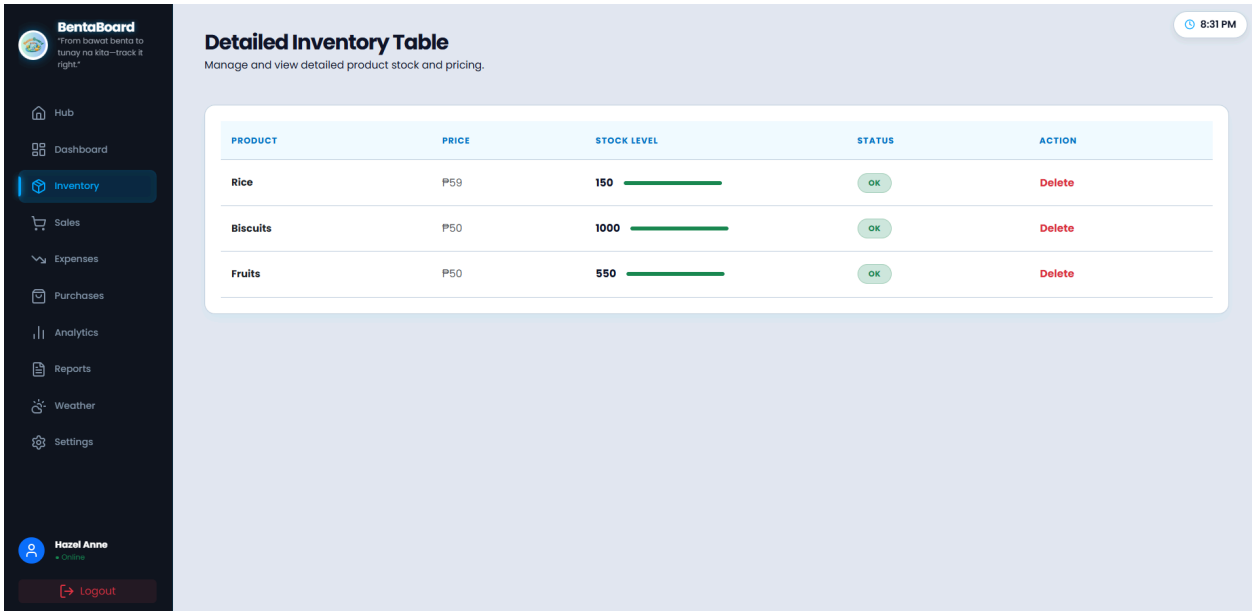


II. Sample backend API response (console or Postman screenshot)

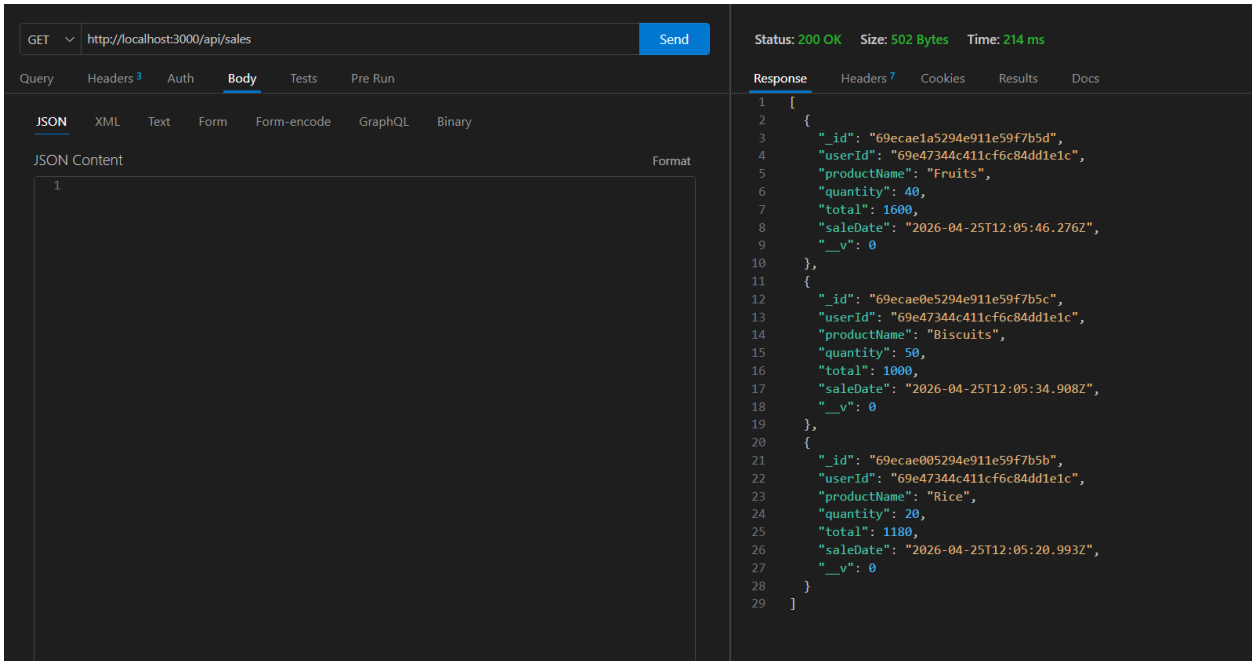
Before displaying data on the website, we had to ensure our backend "GET" routes were sending the correct information. We used tools like Postman/Thunder Client to verify that our routes return clean, valid JSON data that matches our database schema.

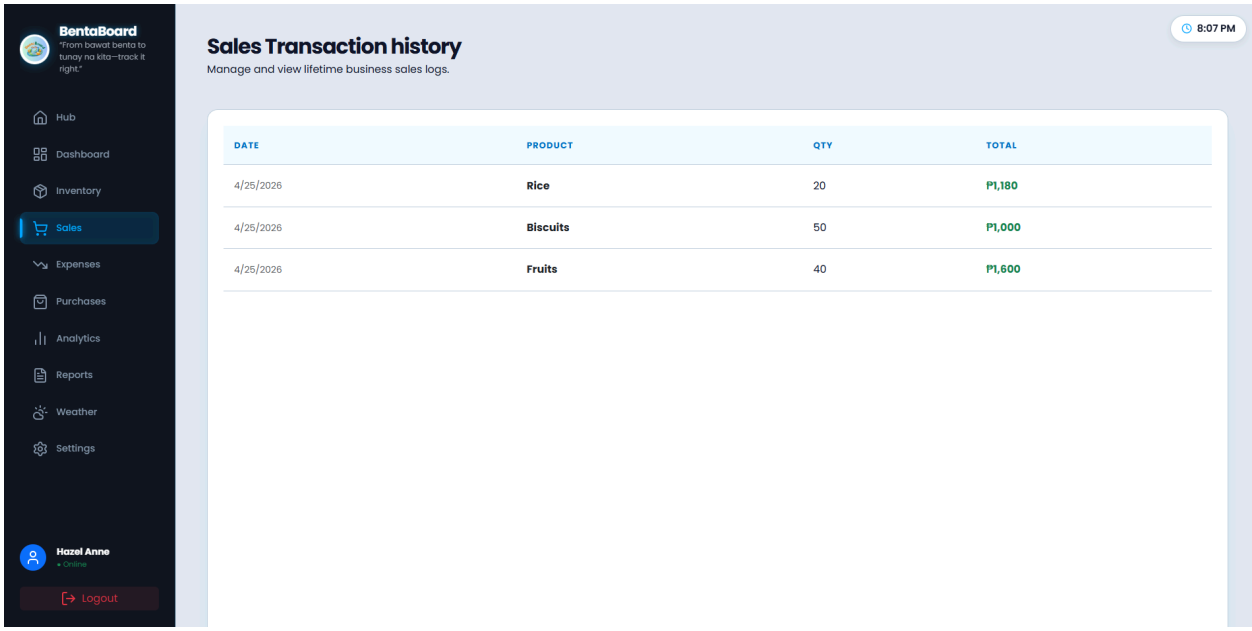
A. GET /api/products - Inventory Product Listing



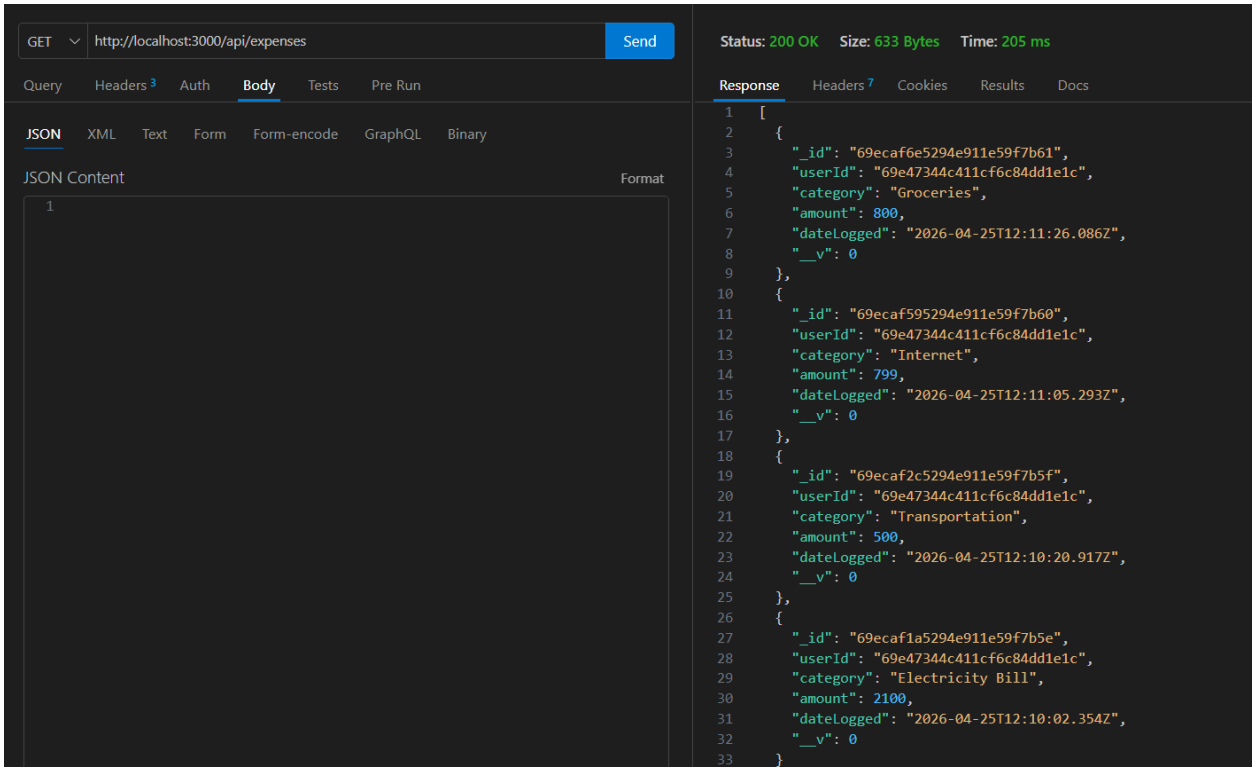


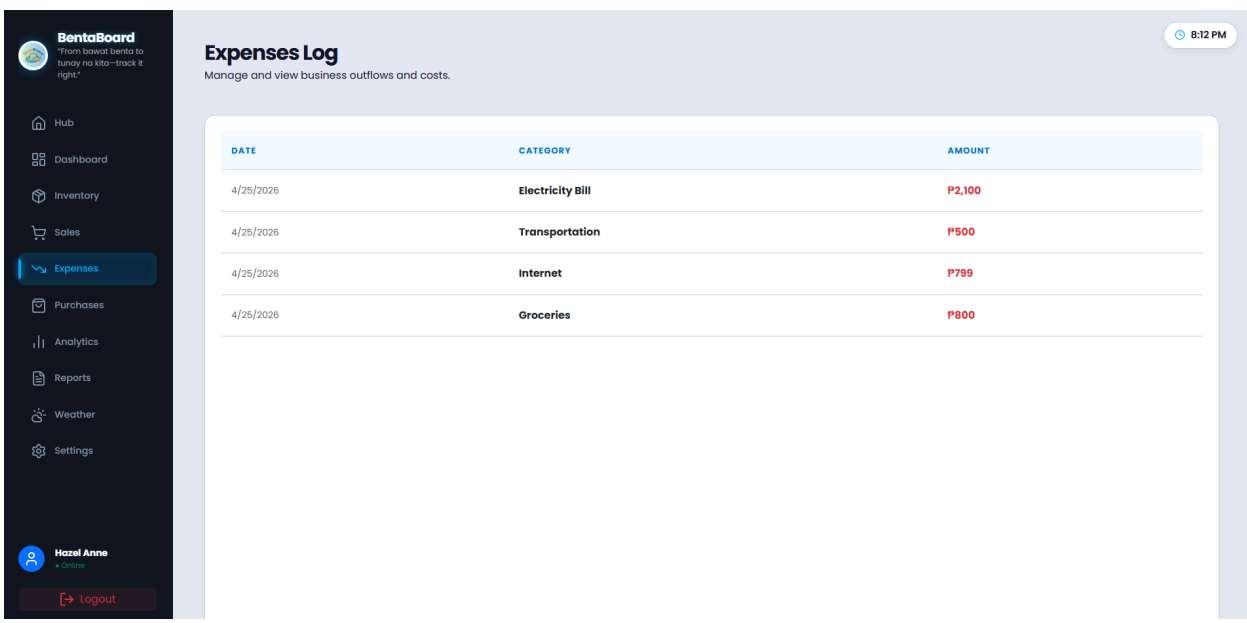
B. **GET /api/sales** - Sales history table



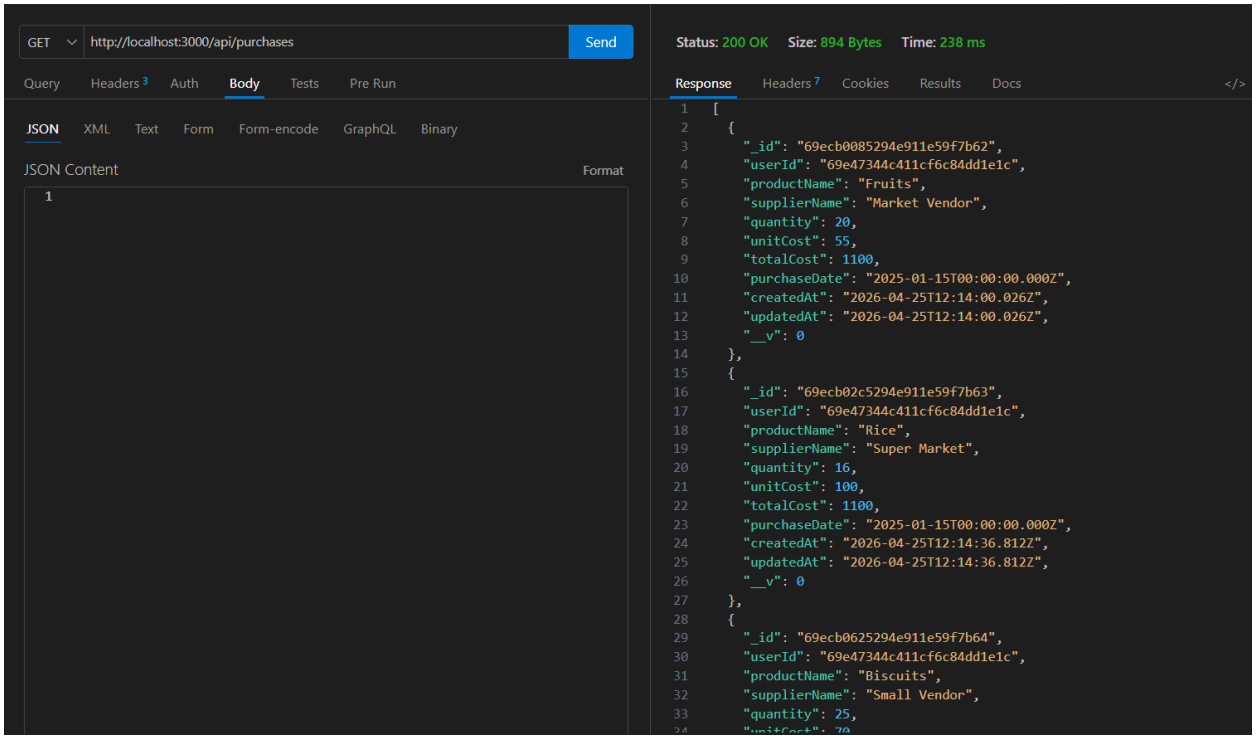


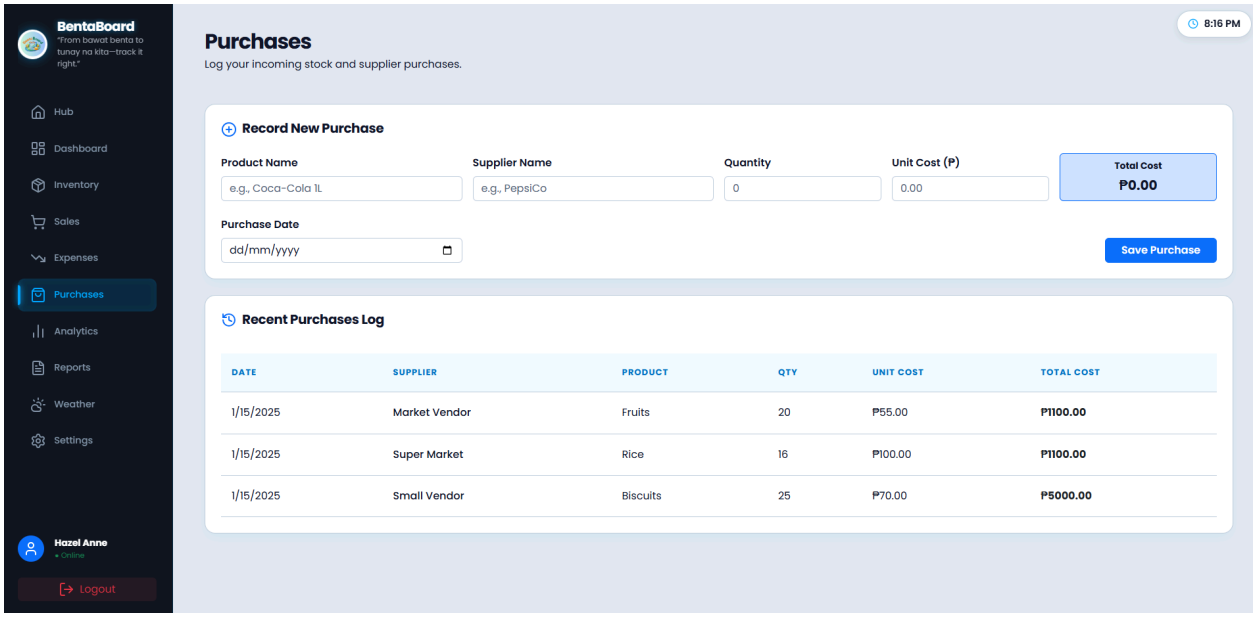
C. **GET /api/expenses** - Expenses log table





D. **GET /api/purchases** - Purchases log table





III. Annotated JavaScript snippet used for rendering data

A. Product Inventory Rendering - inventory.html

```javascript
function renderInventory() {
    const body = document.getElementById('fullInventoryBody');
    if (!body) return;

    const maxStock = 50;

    body.innerHTML = products.map((p, i) => {
        const stockPercent = Math.min((p.stock / maxStock) * 100, 100);
        let barColorClass = 'bg-success';
        let badgeClass    = 'bg-success-subtle text-success border border-success-subtle';
        let statusText    = 'OK';

        if (p.stock <= 5) {
            barColorClass = 'bg-danger';
            badgeClass    = 'bg-danger-subtle text-danger border border-danger-subtle';
            statusText    = 'LOW';
        } else if (p.stock <= 20) {
            barColorClass = 'bg-warning';
        }

        return `
            <tr>
                <td class="fw-bold text-dark" style="font-size: 0.95rem;">${p.name}</td>
                <td class="text-muted">₱${p.price}</td>
                <td>
                    <div class="d-flex align-items-center gap-3">
                        <span class="fw-bold text-dark" style="min-width: 20px;">${p.stock}</span>
                        <div class="progress flex-grow-1" style="height: 6px; max-width: 150px; background-color: var(--border);">
                            <div class="progress-bar ${barColorClass} rounded-pill" role="progressbar" style="width: ${stockPercent}%"></div>
                        </div>
                    </div>
                </td>
                <td>
                    <span class="badge rounded-pill ${badgeClass} px-3 py-2" style="font-size: 0.75rem; letter-spacing: 0.5px;">
                        ${statusText}
                    </span>
                </td>
                <td>
                    <button class="btn btn-link text-danger text-decoration-none fw-bold p-0 m-0 transition" onclick="deleteProduct(${i})">
                        Delete
                    </button>
                </td>
            </tr>
        `;
    }).join('');
}
```

This runs on inventory.html. It checks every product in the products array and builds a table row for each one. It shows the product name, price in pesos, and how many units are left. There's a colored progress bar, the bar fills based on stock level (out of 50 max), and the color changes: green for healthy stock, yellow when it drops below 20, red when it hits 5 or less. A badge next to the bar says either "OK" or "LOW." The last column has a Delete button wired to deleteProduct(i), so sellers can remove items directly from this view.

B. Sales Transaction History Rendering - saleslog.html

```
function renderSalesLog() {
    const body = document.getElementById('salesLogBody');
    if (!body) return;
    body.innerHTML = salesLog.slice().reverse().map(s => `
        <tr>
            <td class="text-muted small">${s.date}</td>
            <td class="fw-bold text-dark">${s.name}</td>
            <td>${s.qty}</td>
            <td class="text-success fw-bold">₱${s.total.toLocaleString()}</td>
        </tr>`).join('');
}
```

This displays every recorded sale on saleslogs.html. It goes through the salesLog array from newest to oldest, so the most recent transaction always sits at the top. Each line prints the date it happened, which product was sold, how many were bought, and how much money the sale brought in. The peso amounts appear in green with proper comma formatting. If there are no sales yet, the table simply stays empty.

C. **Business Expenses Table Rendering** - expenses.html

```
function renderExpenseLog() {
    const body = document.getElementById('expenseLogBody');
    if (!body) return;
    body.innerHTML = expenseLog.slice().reverse().map(e => `
        <tr>
            <td class="text-muted small">${e.date}</td>
            <td class="fw-bold text-dark">${e.category}</td>
            <td class="text-danger fw-bold">₱${e.amount.toLocaleString()}</td>
        </tr>`).join('');
}
```

This works on expenses.html the same way the sales log works, but for costs instead of income. It lists every expense entry , date first, then the category or description of what the money was spent on, and finally the amount in red. Red is used deliberately to visually separate money going out from money coming in. The newest expenses appear first, so the seller sees their latest costs immediately upon opening the page.

D. Revenue Analytics Bar Charts Rendering - analytics.html

```
function renderChart(filter = 'all') {
    const chartCtx = document.getElementById('salesChart');
    if (!chartCtx) return;
    const ctx           = chartCtx.getContext('2d');
    const filteredSales = getFilteredData(salesLog, filter);
    const salesByDate   = {};
    filteredSales.forEach(s => { salesByDate[s.date] = (salesByDate[s.date] || 0) + s.total; });
    const labels     = Object.keys(salesByDate);
    const dataPoints = labels.map(label => salesByDate[label]);
    const isDark     = document.body.classList.contains('dark-theme');
    const gridColor  = isDark ? '#334155' : '#f1f5f9';
    if (salesChartInstance) salesChartInstance.destroy();
    salesChartInstance = new Chart(ctx, {
        type: 'bar',
        data: {
            labels: labels.length > 0 ? labels : ['No Data'],
            datasets: [{
                label: 'Revenue (₱)',
                data:  dataPoints.length > 0 ? dataPoints : [0],
                backgroundColor: '#38bdf8',
                borderRadius: 8,
                barThickness: 45,
                maxBarThickness: 60
            }]
        },
        options: {
            responsive: true,
            maintainAspectRatio: false,
            plugins: { legend: { display: false } },
            scales: {
                y: { beginAtZero: true, grid: { color: gridColor } },
                x: { grid: { display: false } }
            }
        }
    });
}
```

This one is different from the others because it doesn't produce a table . It draws a bar chart. It runs on analytics.html and uses the Chart.js library. Sales data gets grouped by date, and each date becomes a bar on the chart. The height of each bar represents how much revenue was made that day. Before drawing anything, it checks if a chart already exists and removes it, preventing duplicate charts from stacking up. It also accepts a timeframe filter so the buttons on the analytics page can request daily, weekly, monthly, or yearly views, and the chart redraws accordingly. The grid line colors adjust between light and dark mode.

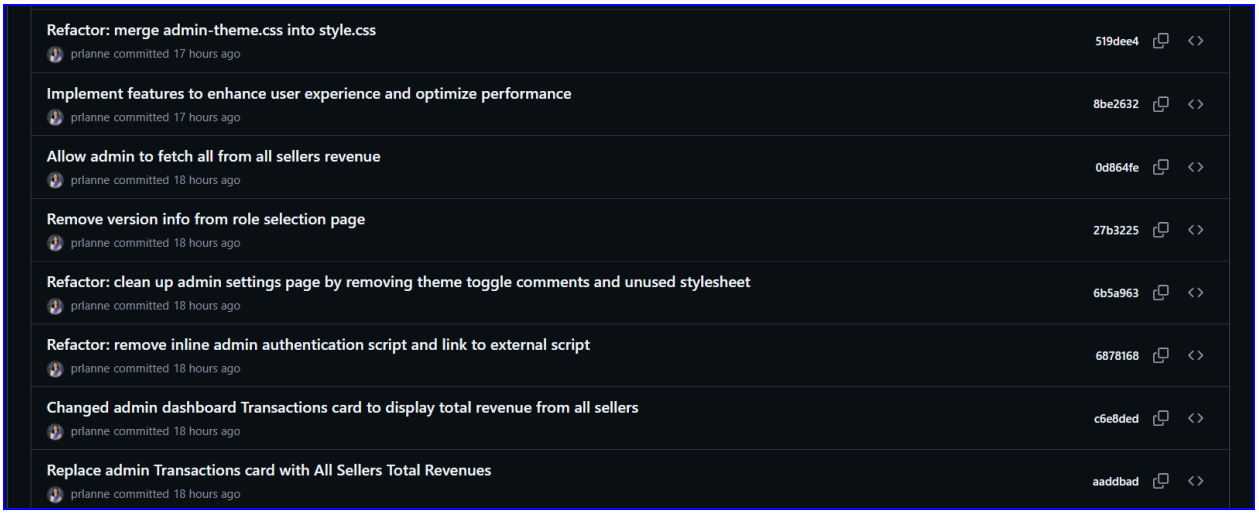
E. Financial Reports & Summary Rendering - reports.html

```
function generateReport(filter = 'all') {
    const filteredSales = getFilteredData(salesLog, filter);
    const filteredExp   = getFilteredData(expenseLog, filter);
    const totalSales    = filteredSales.reduce((sum, s) => sum + s.total, 0);
    const totalExp      = filteredExp.reduce((sum, e) => sum + e.amount, 0);
    const totalItems    = filteredSales.reduce((sum, s) => sum + s.qty, 0);

    if (document.getElementById('reportSalesTotal')) document.getElementById('reportSalesTotal').innerText = `₱${totalSales.toLocaleString()}`;
    if (document.getElementById('reportExpTotal'))   document.getElementById('reportExpTotal').innerText   = `₱${totalExp.toLocaleString()}`;
    if (document.getElementById('reportNet'))        document.getElementById('reportNet').innerText        = `₱${(totalSales - totalExp).toLocaleString()}`;
    if (document.getElementById('reportItems'))      document.getElementById('reportItems').innerText      = totalItems;

    const breakdownBody = document.getElementById('reportBreakdownBody');
    if (breakdownBody) {
        const sortedSales = filteredSales.slice().reverse();
        const sortedExp   = filteredExp.slice().reverse();
        breakdownBody.innerHTML = `
            ${sortedSales.map(s => `
                <tr>
                    <td>${s.date}</td>
                    <td><i data-lucide="shopping-cart" class="text-success me-2" size="14"></i><span class="text-success small fw-bold">Sale</span></td>
                    <td class="fw-bold">${s.name} (Qty: ${s.qty})</td>
                    <td class="text-success fw-bold text-end">₱${s.total.toLocaleString()}</td>
                </tr>`).join('')}
            ${sortedExp.map(e => `
                <tr>
                    <td>${e.date}</td>
                    <td><i data-lucide="trending-down" class="text-danger me-2" size="14"></i><span class="text-danger small fw-bold">Expense</span></td>
                    <td class="fw-bold">${e.category}</td>
                    <td class="text-danger fw-bold text-end">₱${e.amount.toLocaleString()}</td>
                </tr>`).join('')}
            ${sortedSales.length === 0 && sortedExp.length === 0
                ? `<tr><td colspan="4" class="text-center text-muted py-4">No transactions recorded yet.</td></tr>`
                : ''}
        `;
        if (typeof lucide !== 'undefined') lucide.createIcons();
    }
}
```

This handles the entire reports.html page. First, it updates four number cards at the top, total sales, total expenses, items sold, and net profit , by adding up all values in the filtered date range. Then it builds a combined transaction list underneath. Sales and expenses get mixed together, sorted by date with the newest on top. Sales rows appear with a green shopping cart icon and green text. Expense rows show a red trending-down icon and red text. If the date filter returns no results, it displays an empty state message instead. This function also supports the print feature , when a seller clicks print, this is the data that gets printed.

IV. GitHub commit logs

To maintain a professional workflow, our team used organized commits to track the changes made during this phase, specifically tagging our progress for Phase 5.



V. Individual reflections from each group member (3–5 sentences)

Hazel Ann Obidece - Project Manager & Backend Developer

I focused on ensuring that our GET endpoints were optimized and returned the correct data to the frontend. It was challenging to make sure the data structure matched exactly what the frontend expected without causing errors. Seeing the dashboard finally populate with real data from the database was a very rewarding moment for me. This phase helped me understand the importance of data consistency between the server and the client side. I now realize how much coordination is needed to make sure the backend and frontend work as one.

Noe Jr Remolar - Documentation Officer

My role was to document how the data flows from the database to the user's screen through screenshots and clear explanations. I had to carefully verify that the information being displayed on the dashboard was actually accurate to what was stored in MongoDB. This process taught me that even a small error in the data can lead to a very confusing experience for the user. I learned how to use testing tools to confirm that our API was working perfectly before we documented it. It was a great experience seeing our technical work turned into a visual report.

Michael Mantes - Database Manager

I worked on making sure our MongoDB collections were structured in a way that made retrieving data easy and fast. I had to monitor the database closely to ensure that when the frontend requested information, the response was complete and error-free. It was satisfying to see our predefined schemas working perfectly during the data retrieval process. This phase taught me how much the database structure affects the overall speed and performance of a website. I now understand that a well-organized database is the backbone of any successful management system.

Nica Alyssa Racadag - FrontEnd Manager

I was responsible for writing the JavaScript logic that transforms raw JSON data into beautiful and readable cards and tables. The most difficult part was handling "empty states," such as what the user sees when they have no products or sales yet. I spent a lot of time using DOM manipulation to make sure the interface looked professional and organized. This phase taught me how to use the ‘fetch()’ API to create a truly dynamic and interactive user experience. I learned that providing a clean interface is just as important as the data itself.

Pear Anne Odo - GitHub Manager

I managed our repository to ensure that the new frontend display logic and the updated backend routes were merged without any conflicts. I made sure that every major update was tagged clearly so the team could easily track our progress for Phase 5. It was a learning experience to handle multiple changes happening in different folders at the same time. I learned how to resolve small issues that arise when different team members push their code to the main branch. This phase taught me that organized version control is essential for keeping a group project on track.

NOTE:

AI Usage:

*Around 15% of AI assistance has been used on creating this **Phase 5** Lab Report (documentation); to guide us on finishing this report such as, asking assistance when encountering errors and other suggestions, recommendations that would serve as our guide for improvement. Overall, we hereby acknowledge that all the information and key ideas presented on this phase was based on our actual work, outputs, and understanding of the project all throughout implementation from previous and current phase.*